

# TypeScript Practice Problems (Enhanced)

A problem-first revision set for TypeScript fundamentals. These problems are intentionally harder than the bootcamp exercises, but they only assume concepts through the material you said you somewhat understand: types, arrays, tuples, objects, optional properties, functions, unions, unknown, narrowing, type assertions, type aliases, and interfaces.

*This version keeps every original problem exactly as written, but adds an explicit function list (where the original didn't already spell one out) and ready-to-use **test data + console.log calls** for each problem, so you can plug your types/functions in and immediately see if they work. Types are intentionally **not** written for you — that's still your job.*

## Rules

1. Do not look for a tutorial solution first. Try to design the types and functions yourself.
  2. Avoid `any` unless a problem explicitly permits it (none do).
  3. You may use normal JavaScript methods such as `map`, `filter`, `reduce`, `find`, and `sort`.
  4. The goal is not to produce the shortest code. The goal is to make the program type-safe.
  5. When you get stuck, stop and identify the exact TypeScript concept causing the problem.
-

# Difficulty 1 — Warm-up

## 1. Expense Analyzer

Create a type-safe expense analyzer.

Define an `Expense` type with: title, amount, and category. Create an array containing at least 6 expenses. Write functions to: - calculate total spending - find the most expensive expense - return only expenses from a given category - calculate spending for a given category

All functions must have appropriate parameter and return types.

**Functions to implement:** - `calculateTotalSpending(expenses)` - `findMostExpensive(expenses)` - `getExpensesByCategory(expenses, category)` - `calculateCategorySpending(expenses, category)`

**Test data:**

```
const expenses = [
  { title: "Groceries", amount: 42.5, category: "Food" },
  { title: "Netflix", amount: 15, category: "Entertainment" },
  { title: "Bus Pass", amount: 30, category: "Transport" },
  { title: "Dinner Out", amount: 60, category: "Food" },
  { title: "Gym Membership", amount: 25, category: "Health" },
  { title: "Movie Tickets", amount: 20, category: "Entertainment" },
];

console.log(calculateTotalSpending(expenses));
console.log(findMostExpensive(expenses));
console.log(getExpensesByCategory(expenses, "Food"));
console.log(calculateCategorySpending(expenses, "Entertainment"));
```

## 2. Student Results

Define a Student type containing a name and an array of marks. Create at least 5 students. Write functions to: - calculate a student's average - determine whether a student passed (average  $\geq 40$ ) - return the student with the highest average - return all students who passed

Do not duplicate the grade-calculation logic.

**Functions to implement:** - `calculateAverage(student)` - `didPass(student)` - `getTopStudent(students)` - `getPassedStudents(students)`

**Test data:**

```
const students = [
  { name: "Dana", marks: [80, 65, 90] },
  { name: "Rafi", marks: [30, 25, 40] },
  { name: "Mim", marks: [55, 60, 58] },
  { name: "Tanvir", marks: [95, 88, 92] },
  { name: "Oishi", marks: [20, 35, 30] },
];

console.log(calculateAverage(students[0]));
console.log(didPass(students[1]));
console.log(getTopStudent(students));
console.log(getPassedStudents(students));
```

## 3. Product Inventory

Define a Product type with id, name, price, and stock. Create an inventory containing at least 8 products. Implement: -

`findProductById(id)` - `searchProducts(query)` - `getOutOfStockProducts()` - `getLowStockProducts(limit)` - `getTotalInventoryValue()`

`searchProducts` must be case-insensitive. Return an appropriate value when a product cannot be found.

**Test data:**

```
const products = [
  { id: 1, name: "Wireless Mouse", price: 15.99, stock: 20 },
  { id: 2, name: "Mechanical Keyboard", price: 55, stock: 0 },
  { id: 3, name: "USB-C Cable", price: 5.5, stock: 3 },
  { id: 4, name: "Laptop Stand", price: 22, stock: 8 },
  { id: 5, name: "Webcam", price: 40, stock: 0 },
  { id: 6, name: "Monitor Arm", price: 60, stock: 2 },
  { id: 7, name: "Desk Mat", price: 12, stock: 15 },
  { id: 8, name: "Bluetooth Speaker", price: 30, stock: 5 },
];

console.log(findProductById(products, 3));
console.log(findProductById(products, 999));
console.log(searchProducts(products, "keyboard"));
console.log(getOutOfStockProducts(products));
console.log(getLowStockProducts(products, 5));
console.log(getTotalInventoryValue(products));
```

## 4. User Profile

Define a User type with name, age, optional email, and skills (an array of strings). Create several users, including users who do not have an email. Write: - `getUserEmail(user)` , which returns the email or a sensible fallback - `hasSkill(user, skill)` - `getUsersWithSkill(users, skill)` - `getAverageAge(users)`

Handle the optional email safely rather than assuming it exists.

### Test data:

```
const users = [
  { name: "Alif", age: 22, email: "alif@example.com", skills: ["TypeScript", "Node"] },
  { name: "Meher", age: 25, skills: ["React", "CSS"] },
  { name: "Sabbir", age: 19, email: "sabbir@example.com", skills: ["Python"] },
  { name: "Nusrat", age: 30, skills: ["TypeScript", "SQL"] },
];

console.log(getUserEmail(users[1]));
console.log(hasSkill(users[0], "Node"));
console.log(getUsersWithSkill(users, "TypeScript"));
console.log(getAverageAge(users));
```

## 5. Safe Value Processor

Write a function that accepts a value of type unknown. It should return: - the uppercase version if it is a string - the square if it is a number - the number of elements if it is an array - a readable message for anything else

The function must narrow the value before using operations specific to its type.

**Functions to implement:** - `processValue(value)`

### Test data:

```
const testValues = ["hello", 7, [1, 2, 3, 4], true, { a: 1 }, null];

testValues.forEach((v) => console.log(processValue(v)));
```

# Difficulty 2 — Unions and Object Modeling

## 6. Notification System

Create three notification shapes: - email notification: recipient + subject - SMS notification: phone + message - push notification: deviceId + message

Create a union type called Notification. Write `sendNotification(notification)` that prints an appropriate message depending on the notification's available data. Use type narrowing rather than type assertions whenever possible.

**Functions to implement:** - `sendNotification(notification)`

**Test data:**

```
const emailNotif = { kind: "email", recipient: "user@example.com", subject: "Welcome!" };
const smsNotif = { kind: "sms", phone: "01712345678", message: "Your OTP is 4321" };
const pushNotif = { kind: "push", deviceId: "device-99", message: "New login detected" };

console.log(sendNotification(emailNotif));
console.log(sendNotification(smsNotif));
console.log(sendNotification(pushNotif));
```

(Feel free to design the discriminant field however you like — `kind` above is just a suggestion.)

## 7. Payment Processor

Create a PaymentMethod union that allows: - cash - card - mobile banking

Each payment method should have the information needed for that method. For example, a card payment might contain a card number while mobile banking might contain an account number. Write `processPayment(payment)`. The function should calculate the final amount after a method-specific fee: - cash: 0% - card: 2% - mobile banking: 1%

The function must safely determine which payment method it received.

**Functions to implement:** - `processPayment(payment)`

**Test data:**

```
const cashPayment = { type: "cash", amount: 500 };
const cardPayment = { type: "card", amount: 1000, cardNumber: "4111111111111111" };
const mobilePayment = { type: "mobile-banking", amount: 750, accountNumber: "017XXXXXXXX" };

console.log(processPayment(cashPayment));
console.log(processPayment(cardPayment));
console.log(processPayment(mobilePayment));
```

## 8. Shopping Cart

Build the logic for a shopping cart. A cart item should contain a product id, name, price, and quantity. Implement: - `addItem` - `removeItem` - `updateQuantity` - `getSubtotal` - `clearCart`

Adding an existing product should increase its quantity instead of creating a duplicate. A quantity below 1 should not be accepted.

**Test data:**

```

let cart = [];

cart = addItem(cart, { id: 1, name: "Notebook", price: 3.5, quantity: 2 });
cart = addItem(cart, { id: 2, name: "Pen", price: 1, quantity: 5 });
cart = addItem(cart, { id: 1, name: "Notebook", price: 3.5, quantity: 1 }); // should merge into existing item
console.log(cart);

cart = updateQuantity(cart, 2, 10);
console.log(cart);

console.log(getSubtotal(cart));

cart = removeItem(cart, 1);
console.log(cart);

cart = clearCart(cart);
console.log(cart);

```

## 9. Bank Account

Create an Account type with id, owner, and balance. Implement: - `deposit` - `withdraw` - `transfer`

Rules: - deposits and withdrawals cannot be negative - withdrawals cannot exceed the balance - transfers cannot exceed the sender's balance - an account cannot transfer money to itself

Decide what your functions should return when an operation fails. Keep that behavior consistent.

### Test data:

```

const accountA = { id: 1, owner: "Rafi", balance: 1000 };
const accountB = { id: 2, owner: "Dana", balance: 200 };

console.log(deposit(accountA, 500));
console.log(withdraw(accountA, 2000)); // should fail - exceeds balance
console.log(withdraw(accountB, -50)); // should fail - negative amount
console.log(transfer(accountA, accountB, 300));
console.log(transfer(accountA, accountA, 100)); // should fail - same account

```

## 10. Search Result

Create a type-safe search function over an array of users. The function should accept a search string and return users whose names contain that string. Then extend it so the caller can search either by name or by email. Because email is optional, your implementation must safely handle users who do not have one.

**Functions to implement:** - `searchUsersByName(users, query)` - `searchUsers(users, query)` — searches both name and email

### Test data:

```

const users = [
  { name: "Tanvir Ahmed", email: "tanvir@example.com" },
  { name: "Mim Rahman" },
  { name: "Oishi Islam", email: "oishi@mail.com" },
  { name: "Sabbir Hossain" },
];

console.log(searchUsersByName(users, "rah"));
console.log(searchUsers(users, "example.com"));
console.log(searchUsers(users, "mim"));

```

# Difficulty 3 — Real Program Logic

## 11. Todo Engine

Build the logic for a small Todo application. A todo has: - id - title - completed - priority

Priority can only be low, medium, or high. Implement: - `addTodo` - `removeTodo` - `completeTodo` - `uncompleteTodo` - `getPendingTodos` - `getCompletedTodos` - `getTodosByPriority` - `searchTodos`

There is no UI. Focus entirely on the data model and functions.

**Test data:**

```
let todos = [
  { id: 1, title: "Finish TS practice set", completed: false, priority: "high" },
  { id: 2, title: "Grocery shopping", completed: false, priority: "low" },
  { id: 3, title: "Read algorithms chapter", completed: true, priority: "medium" },
];

todos = addTodo(todos, { id: 4, title: "Push bootcamp assignment", completed: false, priority: "high" });
console.log(todos);

todos = completeTodo(todos, 2);
console.log(getCompletedTodos(todos));
console.log(getPendingTodos(todos));

todos = uncompleteTodo(todos, 3);
console.log(getTodosByPriority(todos, "high"));

console.log(searchTodos(todos, "bootcamp"));

todos = removeTodo(todos, 1);
console.log(todos);
```

## 12. Library System

Model a small library. A book has an id, title, author, and whether it is currently available. A member has an id and name. Implement: - `addBook` - `registerMember` - `borrowBook` - `returnBook` - `findAvailableBooks`

A member cannot borrow a book that is already borrowed. Returning an available book should fail safely. Decide what information each operation should return.

**Test data:**

```
let books = [
  { id: 1, title: "Clean Code", author: "Robert Martin", available: true },
  { id: 2, title: "Introduction to Algorithms", author: "CLRS", available: true },
];
let members = [{ id: 1, name: "Nusrat" }];

books = addBook(books, { id: 3, title: "The Pragmatic Programmer", author: "Hunt & Thomas", available: true });
members = registerMember(members, { id: 2, name: "Alif" });

console.log(borrowBook(books, 1, 1)); // Nusrat borrows Clean Code
console.log(borrowBook(books, 1, 2)); // should fail - already borrowed
console.log(returnBook(books, 2, 1)); // should fail - book 2 was never borrowed
console.log(findAvailableBooks(books));
```

## 13. API Response Handler

Model an API response that can either represent success or failure. A successful response contains data. A failed response contains an error message. Write a function that accepts the response and prints the appropriate information. Then use your response type with: - a single User - an array of Products - a single Order

The challenge is to preserve the correct data type instead of falling back to any.

**Functions to implement:** - `handleApiResponse(response)`

**Test data:**

```

const userSuccess = { status: "success", data: { name: "Tanvir", age: 22 } };
const productsSuccess = {
  status: "success",
  data: [
    { id: 1, name: "Mouse", price: 15 },
    { id: 2, name: "Keyboard", price: 55 },
  ],
};
const orderFailure = { status: "error", message: "Order not found" };

console.log(handleApiResponse(userSuccess));
console.log(handleApiResponse(productsSuccess));
console.log(handleApiResponse(orderFailure));

```

## 14. Role-Based Access

Create a user system where a user has a name, an optional email, and a role. Roles are: - admin - editor - viewer

Create functions: - `canDeletePost(user)` - `canEditPost(user)` - `canViewPost(user)`

Then create a function that accepts a role and returns a readable description of what that role can do. Do not use arbitrary strings for roles; make invalid roles impossible at compile time.

**Functions to implement:** - `canDeletePost(user)` - `canEditPost(user)` - `canViewPost(user)` - `describeRole(role)`

**Test data:**

```

const admin = { name: "Sabbir", role: "admin" };
const editor = { name: "Mim", email: "mim@example.com", role: "editor" };
const viewer = { name: "Rafi", role: "viewer" };

console.log(canDeletePost(admin));
console.log(canDeletePost(editor));
console.log(canEditPost(viewer));
console.log(canViewPost(viewer));
console.log(describeRole("editor"));

```

## 15. Order Processing

Create types for Product, OrderItem, and Order. An order contains multiple items and has a customer name. Implement an order-processing function that: - calculates subtotal - applies a discount when appropriate - calculates shipping - produces a final total

Different shipping choices should have different prices. Different discount codes should have different rules. Design the types so invalid shipping choices and discount codes cannot be passed accidentally.

**Functions to implement:** - `calculateSubtotal(order)` - `applyDiscount(subtotal, discountCode)` - `calculateShipping(shippingOption)` - `processOrder(order)` — combines the above into a final total

**Test data:**

```

const order = {
  customerName: "Oishi",
  items: [
    { productId: 1, name: "Notebook", price: 3.5, quantity: 4 },
    { productId: 2, name: "Pen Set", price: 8, quantity: 2 },
  ],
  shippingOption: "express",
  discountCode: "SAVE10",
};

console.log(calculateSubtotal(order));
console.log(applyDiscount(30, "SAVE10"));
console.log(calculateShipping("express"));
console.log(processOrder(order));

```

# Difficulty 4 — Systems

## 16. RPG Inventory

Design the data model for an RPG inventory. Every item has an id and name, but different item categories have different properties: - weapon → damage - armor → defense - potion → healing - quest item → quest id

Create an inventory and implement: - `addItem` - `removeItem` - `findItem` - `useItem` - `getInventoryValue`

When using an item, your code should safely handle the information specific to that item's category.

**Test data:**

```
let inventory = [
  { id: 1, name: "Iron Sword", category: "weapon", damage: 15, value: 50 },
  { id: 2, name: "Leather Armor", category: "armor", defense: 8, value: 30 },
  { id: 3, name: "Health Potion", category: "potion", healing: 25, value: 10 },
  { id: 4, name: "Old Map", category: "quest-item", questId: "find-the-relic", value: 0 },
];

inventory = addItem(inventory, { id: 5, name: "Steel Shield", category: "armor", defense: 12, value: 45 });
console.log(inventory);

console.log(findItem(inventory, 3));
console.log(useItem(inventory, 1)); // use the sword
console.log(useItem(inventory, 3)); // use the potion

console.log(getInventoryValue(inventory));

inventory = removeItem(inventory, 4);
console.log(inventory);
```

## 17. Ability System

Design a small game ability system. Every ability has a name, cooldown, and mana cost. Create at least three different ability shapes, such as: - damage ability - healing ability - movement ability

Write: - `canUseAbility` - `useAbility` - `describeAbility`

Each ability should have at least one property unique to its category. Your functions should safely handle those differences.

**Test data:**

```
const fireball = { name: "Fireball", cooldown: 5, manaCost: 20, category: "damage", damageAmount: 40 };
const heal = { name: "Minor Heal", cooldown: 3, manaCost: 15, category: "healing", healAmount: 25 };
const dash = { name: "Dash", cooldown: 2, manaCost: 5, category: "movement", distance: 10 };

const currentMana = 18;
const abilitiesOnCooldown: string[] = [];

console.log(canUseAbility(fireball, currentMana, abilitiesOnCooldown));
console.log(canUseAbility(heal, currentMana, abilitiesOnCooldown));
console.log(useAbility(heal, currentMana));
console.log(describeAbility(dash));
```

## 18. Mini E-Commerce Engine

Build the core logic of a small e-commerce system. Model: - User - Product - Cart - CartItem - Order

Support: - adding products to a cart - changing quantities - removing items - checking stock - calculating totals - checking out

Checkout should validate stock before creating an order. You do not need a database or UI.

**Functions to implement:** - `addToCart(cart, product, quantity)` - `updateCartItemQuantity(cart, productId, quantity)` - `removeFromCart(cart, productId)` - `checkStock(product, quantity)` - `calculateCartTotal(cart)` - `checkout(cart, products, user)`

**Test data:**



```

const user = { id: 1, name: "Alif" };
const products = [
  { id: 1, name: "Mouse", price: 15, stock: 10 },
  { id: 2, name: "Keyboard", price: 55, stock: 2 },
];

let cart: any[] = [];
cart = addToCart(cart, products[0], 3);
cart = addToCart(cart, products[1], 5); // more than available stock

console.log(checkStock(products[1], 5));
console.log(calculateCartTotal(cart));

cart = updateCartItemQuantity(cart, 1, 1);
console.log(cart);

console.log(checkout(cart, products, user));

```

## 19. Event Logger

Create a type-safe event logger. Support events such as: - LOGIN - LOGOUT - PURCHASE - ERROR

Each event should carry different information. For example, a purchase might contain `userId`, `productId`, and `amount`, while an error contains a message and error code. Create a `logEvent` function that handles each event appropriately.

**Functions to implement:** - `logEvent(event)`

**Test data:**

```

const loginEvent = { type: "LOGIN", userId: 101, timestamp: "2026-08-21T09:00:00Z" };
const logoutEvent = { type: "LOGOUT", userId: 101, timestamp: "2026-08-21T10:30:00Z" };
const purchaseEvent = { type: "PURCHASE", userId: 101, productId: 5, amount: 250 };
const errorEvent = { type: "ERROR", message: "Payment gateway timeout", errorCode: 504 };

[loginEvent, logoutEvent, purchaseEvent, errorEvent].forEach((e) => console.log(logEvent(e)));

```

## 20. Mini Battle Engine

Build a small turn-based battle engine. A character should have: - name - health - attack - defense - speed - abilities

Implement: - `calculateDamage` - `attackTarget` - `useAbility` - `checkDefeated` - `determineTurnOrder` - `startBattle`

Include at least two different ability categories with different data. Keep the project entirely in TypeScript with no UI. The goal is to create a coherent system rather than a collection of isolated functions.

**Test data:**

```
const hero = {
  name: "Hero",
  health: 100,
  attack: 20,
  defense: 8,
  speed: 12,
  abilities: [
    { name: "Power Strike", category: "damage", damageAmount: 30, cooldown: 3 },
    { name: "Second Wind", category: "healing", healAmount: 15, cooldown: 4 },
  ],
};

const goblin = {
  name: "Goblin",
  health: 60,
  attack: 12,
  defense: 4,
  speed: 8,
  abilities: [{ name: "Sneak Attack", category: "damage", damageAmount: 18, cooldown: 2 }],
};

console.log(calculateDamage(hero, goblin));
console.log(attackTarget(hero, goblin));
console.log(useAbility(hero, hero.abilities[1])); // Second Wind
console.log(checkDefeated(goblin));
console.log(determineTurnOrder([hero, goblin]));
console.log(startBattle(hero, goblin));
```

---

---

# Optional Challenge — No Hand-Holding

If the previous 20 start feeling comfortable, take this one without being told which TypeScript features to use.

## 21. Build Your Own Small System

Pick one domain you actually care about: a game mechanic, inventory system, football tournament tracker, F1 race tracker, expense manager, or something else. Design at least 4 related types and build at least 8 functions that operate on them. Include at least one situation where a value can have multiple valid shapes.

Do not start by deciding which TypeScript feature you want to demonstrate. Start with the problem and let the types emerge.

When stuck: do not immediately search for the complete solution. Write down what you know, what TypeScript is complaining about, and what you think the type should be. Then ask for help with that specific concept or error.

**On test data:** this one's by design — since you're picking the domain and the shapes yourself, write your own sample data and your own `console.log(yourFunction(yourData))` calls once your types exist. That's part of the exercise here.

**Suggested path:** Start at Problem 1 and move upward. You do not need to finish every problem before moving on, but Problems 11-20 are intended to make you combine multiple concepts rather than practice one feature at a time.